Rustaceans on The Move
The Global Impact of Rust

Lori Lorusso
Director of Outreach
Rust Foundation

rustfoundation.org

Rust Foundation

Lori Lorusso

Director of Outreach, Rust Foundation
Rust Content Team
CNCF Ambassador



https://www.linkedin.com/in/lorilorusso

About the Rust Foundation

Founded in 2021 the Rust Foundation is an independent non-profit 501(c)(6) organization dedicated to stewarding the Rust programming language, nurturing the Rust ecosystem, and supporting the set of maintainers and contributors governing and developing the Rust Project.

We work closely with the Rust Project, our members AND leaders in the industry/commercial space who have decision-making and budget power.

Rust Foundation

What does that mean?

- We are responsible for ‘keeping the wheels on the bus’ in both infrastructure and core engineering terms
- We are responsible for compliance with multi-jurisdictional laws/regulations
- **We are working to ensuring the health, wellbeing and empowerment of the maintainer community**
- We are responsible for ensuring that there **continues to be a language to steward**





INDUSTRY ADOPTION!!!!

Rust Foundation

Why is Rust Appealing to Industry Adopters?

Let’s break it down…

- 🔒 Security! Eliminate memory bugs on day 1.
- ☁️ Performance at scale, no garbage collector pauses.
- 🏭 No runtime, deterministic behavior.
- 🛠️ Maintainability, fearless refactoring.

THE ADOPTION LANDSCAPE

Rust is moving from pioneering projects to mainstream enterprise infrastructure. Here are a few examples by industry…

☁️ Cloud & Infrastructure

AWS, Google Cloud, Cloudflare, Microsoft Azure

🔒 Security & OS

Linux Kernel, Android, NSA guidelines

📱 Consumer Platforms

Meta/WhatsApp, Discord, Dropbox, 1Password

🏭 Safety-Critical / Embedded

Ferrous Systems (Ferrocene), automotive ISO-26262, avionics DO-178

🛠️ Developer Tooling

JetBrains (RustRover), Huawei (rustdoc, docs.rs), Mainmatter

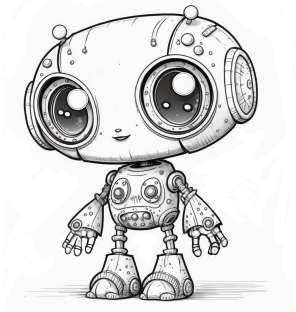
🌐 Enterprise Consulting

Red Badger — Crux pattern for iOS/Android/Web code sharing

What does Industry adoption mean...

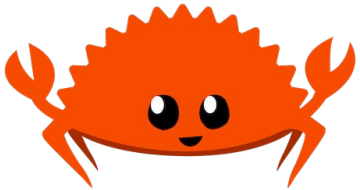
- 🔧 **New tooling**: IDEs, debuggers, profilers, certified compilers...
- 📦 **Reliability**: Testing Rust in the real world and creating new user stories
- 🌍 **Standardization**: Rust has entered automotive, aerospace, medical devices, and needs formal specifications, safety certifications, and regulatory recognition
- 💸 **Funding:** Financial support of Rust project maintainers and directed funds to new initiatives

Rust = Security







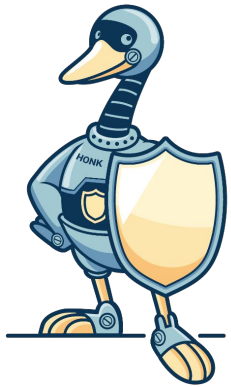






Ai

In 2022 Rust was identified as a "Critical Open Source Software Project"



OpenSSF

OPEN SOURCE SECURITY FOUNDATION

Rust Foundation's Security Initiative



- Threat modeling and ecosystem scanning work
- Implementing Trusted Publishing for crates.io
- Working on infrastructure needed for crate signing
- Continues to strengthen tooling to support addressing security vulnerabilities



2025 Technology Report

How is the Industry actively supporting security efforts?

CASE STUDY

Ferrocene: The First Safety-Qualified Rust Toolchain

October 2023



Certified for:

- **ISO 26262 ASIL D**
  Automotive: highest hazard classification
- **IEC 61508 SIL 3**
  Industrial safety systems
- **IEC 62304 Class C**
  Medical devices
- **DO-178C DAL C support**
  Avionics certification path



"[Rust] has a good feel for when innovation is needed or an existing standard should be taken and implemented well."

- *Florian Gilcher, Founder, Ferrous Systems*

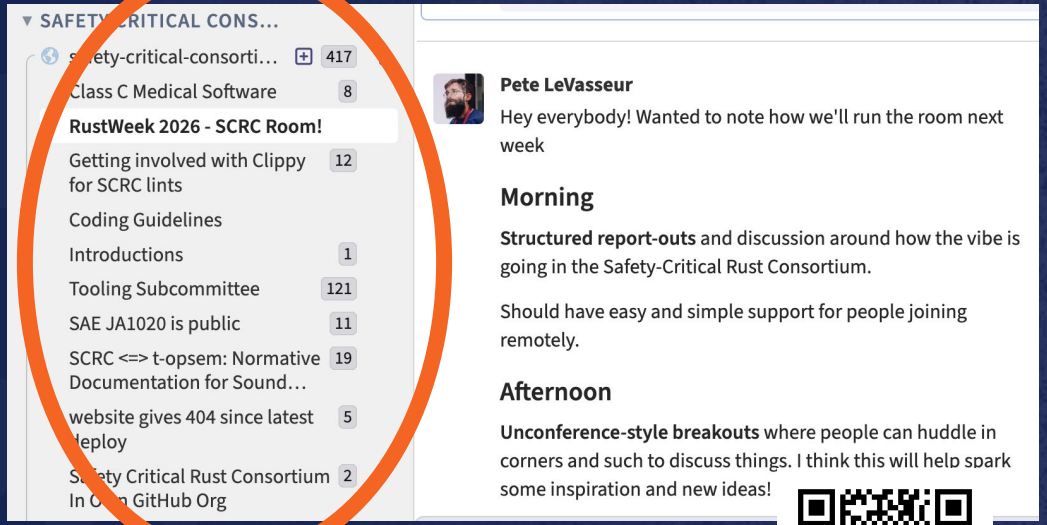
Safety-Critical Rust Consortium

Supporting the responsible use of Rust in systems whose failure can impact human life or cause severe environmental or property harm.

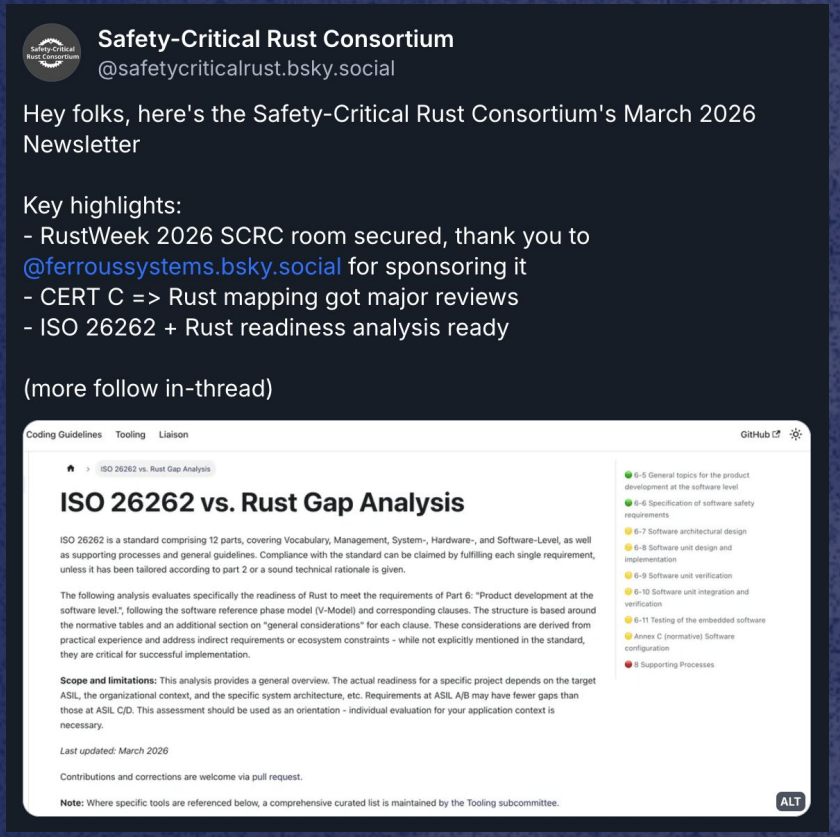
- Began in 2024 with 10 founding organizations.
- Over 100 members, from companies and individuals
- Focusing on coding guidelines, tooling and working across industry groups
- Open to join



Zulip Channel



Bluesky



Are We Safety Critical Yet?
ISO 26262 vs. Rust Gap Analysis

ISO 26262 vs. Rust Gap Analysis

ISO 26262 is a standard comprising 12 parts, covering Vocabulary, Management, System-, Hardware-, and Software-Level, as well as supporting processes and general guidelines. Compliance with the standard can be claimed by fulfilling each single requirement, unless it has been tailored according to part 2 or a sound technical rationale is given.

The following analysis evaluates specifically the readiness of Rust to meet the requirements of Part 6: "Product development at the software level.", following the software reference phase model (V-Model) and corresponding clauses. The structure is based around the normative tables and an additional section on "general considerations" for each clause. These considerations are derived from practical experience and address indirect requirements or ecosystem constraints - while not explicitly mentioned in the standard, they are critical for successful implementation.

Scope and limitations: This analysis provides a general overview. The actual readiness for a specific project depends on the target ASIL, the organizational context, and the specific system architecture, etc. Requirements at ASIL A/B may have fewer gaps than those at ASIL C/D. This assessment should be used as an orientation - individual evaluation for your application context is necessary.

Last updated: March 2026

Contributions and corrections are welcome via pull request.

Note: Where specific tools are referenced below, a comprehensive curated list is maintained by the Tooling subcommittee.

🟢 6-5 General topics for the product development at the software level
🟢 6-6 Specification of software safety requirements
🟡 6-7 Software architectural design
🟡 6-8 Software unit design and implementation
🟡 6-9 Software unit verification
🟡 6-10 Software unit integration and verification
🟡 6-11 Testing of the embedded software
🟡 Annex C (normative) Software configuration
🔴 8 Supporting Processes

https://arewesafetycriticalyet.org/

French Cybersecurity Agency (ANSSI)



“The guide intends to group recommendations that should be applied for application development with strong security level requirements.”

Checklist

Checklist

- Development environment:
  - Rule - Use a stable compilation toolchain (DENV-STABLE)
  - Rule - Exclusive use of tier 1 of `rustc` for safety-critical software (DENV-TIERS)
  - Rule - Track Cargo.lock in version control system (DENV-CARGO-LOCK)
  - Rule - Keep default values for critical variables in cargo profiles (DENV-CARGO-OPTS)
  - Rule - Keep default values for compiler environment variables when running cargo (DENV-CARGO-ENVVARS)
  - Recommendation - Use Rust formatter (rustfmt) (DENV-FORMAT)
  - Rule - Use linter regularly (DENV-LINTER)
  - Rule - Manually check automatic fixes (DENV-AUTOFIX)
- Libraries:
  - Rule - Validation of Direct Third-Party Dependencies (LIBS-VETTING-DIRECT)
  - Recommendation - Validation of Transitive Third-Party Dependencies (LIBS-VET TRANSITIVE)
  - Rule - Check for outdated dependencies (cargo-outdated) (LIBS-OUTDATED)
  - Rule - Check for security vulnerability reports on dependencies (cargo-audit) (L AUDIT)

bitly

Rust in Robotics

C/C++, Python, Java & Matlab



Current Issues w/Languages

- Memory safety
- Race Conditions
- Undefined Behavior
- Real-time efficiency

Rust for Robotics: Safer, Faster Systems for Autonomous Applications

Where does Rust fit in the robotics ecosystem?



- Rust can fit in all stages!
- Replace the critical 20% that cause 80% of inefficiency / unsafeness



Examples

The Safety Story

Zenoh: Zero Overhead Network Protocol

Tier-1 support RMW in ROS 2 Kilted!

Fearless concurrency ensure safety



Copper-rs: Robotics runtime

Copper is to robots what a game engine is to games





Modernization of Infrastructure

Rust Foundation

CASE STUDY

Telecommunications

WhatsApp (Meta)

What they did:
WhatsApp rebuilt its core media parsing components in Rust, 160,000 lines of C++ down to 90,000 lines while adding memory safety protections. Media parsing is historically high-risk for memory vulnerabilities.

Why Rust:
Strong memory safety guarantees without sacrificing performance or cross-platform reach. This was a deliberate, long-term architectural decision — not an experiment.

Impact:
Now protecting billions of users across multiple platforms. Memory safety treated as foundational to modern security engineering, not optional.

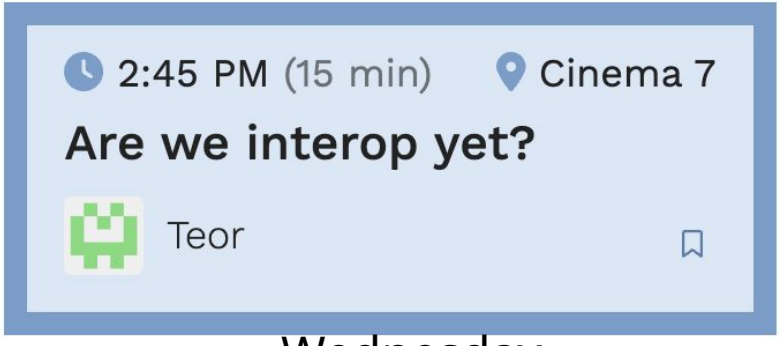
One of the largest Rust client-side deployments in production

2B+

users protected by Rust-rebuilt media parser (2026)

C++/Rust Interop Initiative

- Funded through a generous donation from Google
- C++/Rust Interop Problem Space Mapping



Wednesday

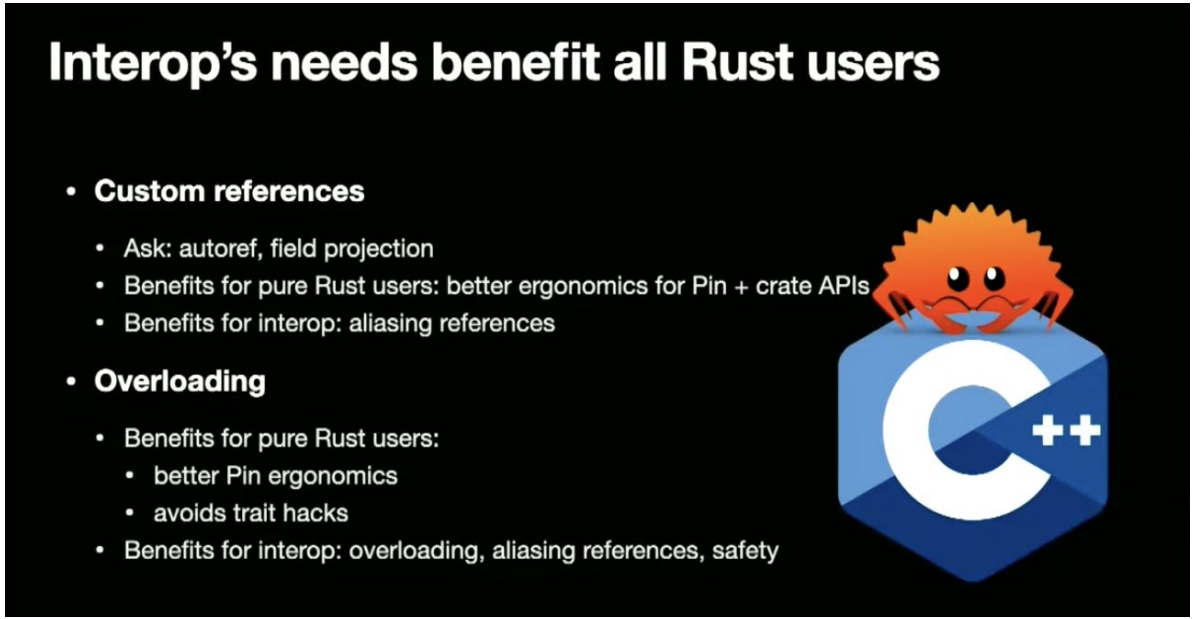
C++/Rust Interop Problem Space Mapping #388

Specifically, the last month we've made detailed progress on two high-priority use cases:

- calling an overloaded C++ function from Rust, with a Rust lang experiment - implementation discussion
  - we've merged a refactor to prepare for this experiment, which gave some nice perf wins
  - the initial overloading experiment PR has been through two rounds of review, and is waiting for my revisions and rebasing
- adding Rust to an existing C++ build system
  - Outreachy applicants wrote interop example code PRs
  - these interop user experiences are waiting for analysis, so they can be summarised in the build system and overloading problem statements
  - this will likely happen after RustWeek and the All Hands



"Fine-Grained C++ Interop"



bitly

Tyler Mandry & Taylor Cramer, Google

CASE STUDY

AWS · Amazon Web Services (Platinum Member)

Why AWS bets on Rust

AWS uses Rust in critical infrastructure because its out-of-the-box safety features reduce time-to-security while delivering C/C++-level performance.

“[Rust] code was 10x faster than our carefully tuned Kotlin implementation despite no attempt to make it faster.”
- Werner Vogels, CTO, AWS (at RE:Invent 2025)

Strategic commitment
AWS employs several core Rust & Tokio maintainers full-time. Rust is described as a "critical component of [AWS’] long-term strategy", covering developer tools, infrastructure, interoperability, and formal verification.

Rust-Powered AWS Products

- Firecracker VMM (powers Lambda)
- Amazon S3
- Amazon EC2
- Amazon CloudFront
- Route 53
- Bottlerocket (container OS)
- AWS Nitro System + Nitro Enclaves
- Tokio async runtime (maintained)



Rust for Linux

Rust for Linux is the project adding support for the Rust language to the Linux kernel.

- Independent project that directly liaises with the Rust Project
- Attends the Rust All-Hands
- Invites Project members to Kangrejos

CASE STUDY

Canonical (Gold Member)

Replacing foundational Linux utilities with Rust carefully, but purposefully.

The goal

Starting with Ubuntu 25.10, Canonical is replacing core GNU utilities (*ls, cp, mv, sudo* and more) with Rust reimplementations — targeting the world's most deployed Linux distribution.

Why Rust specifically

*"Performance is high on my list of priorities — but it's not the primary driver. It's the enhanced **resilience and safety** that is most attractive. If foundational software fails, so do all the layers built on top."*
— Jon Seager, VP Engineering, Canonical

Rust replacements in scope

- → uutils coreutils (ls, cp, mv...)
- → uutils findutils
- → uutils diffutils
- → sudo-rs (Trifecta Tech Foundation)

Roadmap

25.10	uutils coreutils becomes default
26.04 LTS	Full rollout if conditions are right

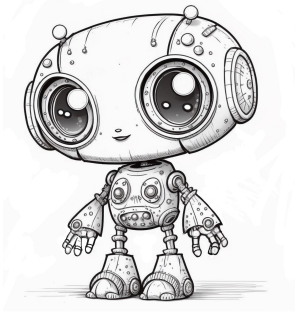
Ubuntu runs on millions of devices worldwide.

A Pattern Emerges

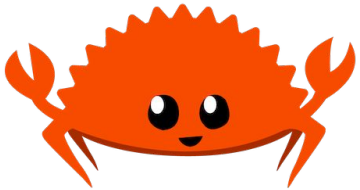


Rust = EVERYWHERE















WHAT INDUSTRY ADOPTERS HAVE IN COMMON

01

Safety is the primary driver

Case studies cite memory safety as the #1 motivation. This eliminates classes of bugs that C/C++ fundamentally cannot prevent.

02

No performance penalty

Rust delivers C/C++-level runtime speed. AWS, Google, and Meta all confirm that safety no longer requires sacrificing performance.

03

Cross-domain versatility

Rust powers OS kernels (Linux), messaging apps (WhatsApp), cloud infrastructure (AWS), safety-critical embedded (Ferrous Systems), and web apps (Mainmatter).

04

Talent and culture shift

AWS, Google, and others report that Rust attracts engineers passionate about quality. Better code review, higher confidence, and a welcoming community.

05

Strategic, not experimental

Adopters often describe Rust as a long-term architectural decision. Red Badger: "There are no longer advantages in waiting."

06

Community investment pays off

Companies that invest in Rust (Foundation membership, hiring maintainers, sponsoring research, etc.) get more secure systems, better tooling, and competitive advantage.



Newest Initiative...
Rust Commercial Network

- Starting working groups to focus on industry-specific challenges
- Improving visibility into Rust adoption
- Identifying unmet needs of Rust users
- Offering feedback on the usability, reliability, and performance of Rust and the ecosystem
- Developing pathways to contributions within the Rust Project and Ecosystem
- Collaborating with the Rust Project





Thank you!

rustfoundation.org/get-involved

rustfoundation.org